

TodoApp — API Specification

API Specification · Version v1

Base URL (Production): `https://your-api.azurewebsites.net`

Base URL (Local): `https://localhost:5240`

Content-Type: `application/json`

API Version: v1 (no prefix — routes start with `/api/`)

Authentication

The API uses **JWT Bearer tokens**. After login, include the token in every protected request:

```
Authorization: Bearer <access_token>
```

Access tokens expire in **60 minutes**. Use the refresh endpoint to obtain a new pair without re-authenticating.

Token Lifecycle

```
POST /api/Auth/register → { token, refreshToken }
POST /api/Auth/login    → { token, refreshToken } (or 2FA challenge)
POST /api/Auth/login-2fa → { token, refreshToken }
POST /api/Auth/refresh  → { token, refreshToken } (old refreshToken is revoked)
POST /api/Auth/logout   → 204 (refreshToken revoked)
```

SignalR (WebSocket)

WebSocket connections cannot set `Authorization` headers. Pass the token as a query parameter:

```
wss://your-api.azurewebsites.net/hubs/todo?access_token=<access_token>
```

Error Format (RFC 7807)

All error responses follow this structure:

```
{
  "type": "https://tools.ietf.org/html/rfc7231#section-6.5.1",
  "title": "Validation Error",
  "status": 400,
  "detail": "One or more validation errors occurred.",
  "errors": {
    "email": ["Must be a valid email address."],
    "password": ["Minimum 8 characters required."]
  }
}
```

Status	Meaning
400	Validation error — check <code>errors</code> field
401	Missing or expired token
403	Authenticated but not authorized for this resource
404	Resource not found (or access denied, to prevent enumeration)
409	Conflict (e.g., duplicate email, duplicate share)
429	Rate limit exceeded

Status	Meaning
500	Unexpected server error

Shared Response Wrappers

PaginatedResponse<T>

```
{
  "items": [],
  "page": 1,
  "pageSize": 20,
  "totalCount": 45,
  "totalPages": 3,
  "hasNextPage": true,
  "hasPreviousPage": false
}
```

CollectionResponse<T>

```
{
  "items": []
}
```

Enums

Priority

Value	Integer
Low	0
Medium	1
High	2
Urgent	3

Status (*TodoItem*)

Value	Integer
Open	0
Completed	1

TaskFilterType

Value	Integer	Description
All	0	Owned + shared (default)
OnlyMine	1	Only tasks I own
SharedWithMe	2	Only tasks shared with me
SharedByMe	3	Tasks I own and have shared

TransferRequestStatus

Value	Description
Pending	Awaiting recipient response
Accepted	Ownership transferred
Rejected	Recipient declined
Cancelled	Sender cancelled

Endpoints

Auth

POST /api/Auth/register

Auth: Public · Rate limit: 3/min

Request body:

```
{
  "email": "user@example.com",
  "password": "Password123!"
}
```

Response 200:

```
{
  "userId": "4e8d5ea2-3c12-4f89-8d7b-123456789abc",
  "email": "user@example.com",
  "token": "<jwt>",
  "refreshToken": "<refresh_token>",
  "requiresTwoFactor": false,
  "twoFactorToken": null
}
```

POST /api/Auth/login

Auth: Public · Rate limit: 5/min

Request body:

```
{
  "email": "user@example.com",
  "password": "Password123!"
}
```

Response 200 — standard:

```
{
  "userId": "4e8d5ea2-3c12-4f89-8d7b-123456789abc",
  "email": "user@example.com",
  "token": "<jwt>",
  "refreshToken": "<refresh_token>",
  "requiresTwoFactor": false,
  "twoFactorToken": null
}
```

Response 200 — 2FA required:

```
{
  "userId": null,
  "email": null,
  "token": null,
  "refreshToken": null,
  "requiresTwoFactor": true,
  "twoFactorToken": "<temporary_2fa_token>"
}
```

When `requiresTwoFactor` is `true`, call `POST /api/Auth/login-2fa` with the `twoFactorToken` and the TOTP code.

POST /api/Auth/login-2fa

Auth: Public

Request body:

```
{
  "twoFactorToken": "<temporary_2fa_token>",
  "code": "123456"
}
```

Response 200: Same as standard `AuthResponse` above (with real `token` and `refreshToken`).

POST /api/Auth/refresh

Auth: Public

Request body:

```
{
  "refreshToken": "<refresh_token>"
}
```

Response 200:

```
{
  "userId": "4e8d5ea2-3c12-4f89-8d7b-123456789abc",
  "email": "user@example.com",
  "token": "<new_jwt>",
  "refreshToken": "<new_refresh_token>",
  "requiresTwoFactor": false,
  "twoFactorToken": null
}
```

The old `refreshToken` is immediately revoked. Always store and use the latest pair.

POST /api/Auth/logout

Auth: Public (token optional)

Request body:

```
{
  "refreshToken": "<refresh_token>"
}
```

Response 204: No content.

POST /api/Auth/forgot-password

Auth: Public · **Rate limit:** 2/min

Request body:

```
{
  "email": "user@example.com"
}
```

Response 200:

```
{
  "message": "If this email is registered, a password reset link has been sent."
}
```

Response is identical whether the email exists or not (user enumeration prevention).

POST /api/Auth/reset-password

Auth: Public

Request body:

```
{
  "token": "<url_decoded_reset_token>",
  "newPassword": "NewPassword123!"
}
```

The reset link in the email contains a URL-encoded token. When building the reset screen, URL-decode the token from the deep link query parameter before sending it in this request body.

Response 200:

```
{
  "message": "Password has been successfully changed."
}
```

PUT /api/Auth/change-password

Auth: Required

Request body:

```
{
  "currentPassword": "OldPassword123!",
  "newPassword": "NewPassword123!"
}
```

Response 204: No content.

POST /api/Auth/2fa/enable

Auth: Required

Request body: None.

Response 200:

```
{
  "secret": "JBSWY3DPEHPK3PXP",
  "qrCodeUri": "otpauth://totp/ToDoApp:user@example.com?secret=JBSWY3DPEHPK3PXP&issuer=ToDoApp"
}
```

Show `qrCodeUri` as a QR code or let the user enter `secret` manually into their authenticator app. Then call `POST /api/Auth/2fa/verify` to complete activation.

POST /api/Auth/2fa/verify

Auth: Required

Request body:

```
{
  "code": "123456"
}
```

Response 200:

```
{
```

```
{
  "message": "Two-factor authentication successfully activated."
}
```

POST /api/Auth/2fa/disable

Auth: Required

Request body:

```
{
  "code": "123456"
}
```

Response 200:

```
{
  "message": "Two-factor authentication has been disabled."
}
```

Todo Items

POST /api/ToDoItems

Auth: Required

Request body:

```
{
  "title": "Backend Architecture Report",
  "description": "Prepare Clean Architecture documentation",
  "dueDate": "2026-10-01T18:00:00Z",
  "priority": 2,
  "todoListId": "b1a2c3d4-e5f6-7a8b-9c0d-112233445566"
}
```

Field	Type	Required	Notes
title	string	■	Max 200 chars
description	string?	■	
dueDate	datetime?	■	ISO 8601 UTC
priority	int	■	Default: 1 (Medium)
todoListId	guid?	■	Must belong to the user

Response 201: `ToDoItemResponse` (see schema below).

GET /api/ToDoItems

Auth: Required

Query parameters:

Parameter	Type	Default	Notes
filterType	int	0	0=All, 1=OnlyMine, 2=SharedWithMe, 3=SharedByMe
search	string?	—	Searches title and description
status	int?	—	0=Open, 1=Completed

Parameter	Type	Default	Notes
priority	int?	—	0=Low, 1=Medium, 2=High, 3=Urgent
todoListId	guid?	—	Filter by list
dueDateFrom	datetime?	—	ISO 8601 UTC
dueDateTo	datetime?	—	ISO 8601 UTC
sortBy	string?	"createdAt"	createdAt, dueDate, title, priority
sortOrder	string?	"desc"	asc, desc
page	int	1	
pageSize	int	20	Max: 100

Response 200: `PaginatedResponse<TodoItemResponse>`

GET `/api/TodoItems/{id}`

Auth: Required (owner or shared user)

Response 200: `TodoItemResponse` (includes `subTasks`, `tags`, `sharedWith`).

PUT `/api/TodoItems/{id}`

Auth: Required (owner only)

Request body:

```
{
  "title": "Updated Title",
  "description": "Updated description",
  "dueDate": "2026-11-01T09:00:00Z",
  "priority": 3,
  "todoListId": "b1a2c3d4-e5f6-7a8b-9c0d-112233445566"
}
```

Response 200: Updated `TodoItemResponse`.

PATCH `/api/TodoItems/{id}/complete`

Auth: Required (owner only)

Request body: None.

Response 200: Updated `TodoItemResponse` (status toggled between `Open` ↔ `Completed`).

DELETE `/api/TodoItems/{id}`

Auth: Required (owner only)

Moves the item to trash (soft delete).

Response 204: No content.

DELETE /api/ToDoItems/{id}/permanent

Auth: Required (owner only, item must be in trash)

Permanently deletes the item.

Response 204: No content.

POST /api/ToDoItems/{id}/restore

Auth: Required (owner only)

Restores an item from trash.

Response 200: Restored `ToDoItemResponse`.

GET /api/ToDoItems/trash

Auth: Required

Query parameters: `page`, `pageSize` (same defaults as list endpoint).

Response 200: `PaginatedResponse<ToDoItemResponse>`

GET /api/ToDoItems/{id}/activities

Auth: Required (owner or shared user)

Response 200: `CollectionResponse<ToDoItemActivityResponse>`

Subtasks

POST /api/todoitems/{taskId}/subtasks

Auth: Required (owner or shared user)

Request body:

```
{
  "title": "Write unit tests"
}
```

Response 201: `SubTaskResponse`

GET /api/todoitems/{taskId}/subtasks

Auth: Required (owner or shared user)

Response 200: `CollectionResponse<SubTaskResponse>`

PATCH /api/subtasks/{id}/complete

Auth: Required (owner or shared user)

Request body: None.

Response 200: Updated `SubTaskResponse` (status toggled).

DELETE /api/subtasks/{id}

Auth: Required (owner only)

Response 204: No content.

Tags

GET /api/tags

Auth: Required

Response 200: `CollectionResponse<TagResponse>`

POST /api/tags

Auth: Required · **Role:** Admin

Request body:

```
{
  "name": "Frontend"
}
```

Response 201: `TagResponse`

GET /api/tags/{tagId}/todoitems

Auth: Required

Response 200: `PaginatedResponse<TodoItemResponse>`

GET /api/todoitems/{taskId}/tags

Auth: Required (owner or shared user)

Response 200: `CollectionResponse<TagResponse>`

POST /api/todoitems/{taskId}/tags/{tagId}

Auth: Required (owner)

Attaches a tag to a task (idempotent).

Request body: None.

Response 200:

```
{
  "message": "Tag successfully attached."
}
```

DELETE /api/todoitems/{taskId}/tags/{tagId}

Auth: Required (owner)

Detaches a tag from a task. The tag is not deleted from the system.

Response 204: No content.

Todo Lists

POST /api/TodoLists

Auth: Required

Request body:

```
{
  "name": "Work Projects",
  "colorCode": "#FF5733"
}
```

Field	Type	Required	Notes
name	string	■	Max 100 chars
colorCode	string?	■	Hex color, e.g. #FF5733 (max 7 chars)

Response 201: TodoListResponse

GET /api/TodoLists

Auth: Required

Response 200: CollectionResponse<TodoListResponse>

GET /api/TodoLists/{id}

Auth: Required

Response 200: TodoListResponse

PUT /api/TodoLists/{id}

Auth: Required (owner)

Request body:

```
{
  "name": "Personal Growth",
  "colorCode": "#33C1FF"
}
```

Response 200: Updated TodoListResponse

DELETE /api/TodoLists/{id}

Auth: Required (owner)

Response 204: No content.

Task Sharing

POST /api/todoitems/{taskId}/shares

Auth: Required (owner only)

Request body:

```
{
  "email": "friend@example.com"
}
```

Response 200:

```
{
  "message": "Task successfully shared."
}
```

GET /api/todoitems/{taskId}/shares

Auth: Required (owner or shared user)

Response 200: `CollectionResponse<SharedUserResponse>`

DELETE /api/todoitems/{taskId}/shares/{userId}

Auth: Required (owner only)

Removes a specific user from the task's share list.

Response 204: No content.

DELETE /api/todoitems/{taskId}/shares/me

Auth: Required (shared user)

The current user voluntarily leaves a shared task.

Response 204: No content.

Ownership Transfer

POST /api/todoitems/{taskId}/transfer-requests

Auth: Required (owner only)

Request body:

```
{
  "newOwnerEmail": "new_owner@example.com"
}
```

Response 200: `TransferRequestResponse`

GET /api/transfer-requests/pending

Auth: Required

Returns transfer requests where the current user is the recipient.

Response 200: `CollectionResponse<TransferRequestResponse>`

POST /api/transfer-requests/{requestId}/accept

Auth: Required (recipient only)

Request body: None.

Response 200:

```
{
  "message": "Transfer request accepted and ownership transferred."
}
```

POST /api/transfer-requests/{requestId}/reject

Auth: Required (recipient only)

Request body: None.

Response 200:

```
{
  "message": "Transfer request rejected."
}
```

POST /api/transfer-requests/{requestId}/cancel

Auth: Required (original owner)

Request body: None.

Response 200:

```
{
  "message": "Transfer request cancelled."
}
```

User Account

GET /api/Users/me

Auth: Required

Response 200:

```
{
  "userId": "4e8d5ea2-3c12-4f89-8d7b-123456789abc",
  "email": "user@example.com",
  "role": "User",
  "isTwoFactorEnabled": false
}
```

DELETE /api/Users/me

Auth: Required

Permanently deletes the account and all associated data. This action is irreversible.

Request body:

```
{
  "password": "Password123!"
}
```

Response 204: No content.

Health Checks

GET /health

Auth: Public

Returns **200 OK** if the API process is alive.

GET /health/ready

Auth: Public

Returns **200 OK** if the API can reach the database.

Real-Time Notifications (SignalR)

Hub URL: /hubs/todo

Protocol: SignalR (WebSocket with Long Polling fallback)

Authentication: Pass the JWT as **?access_token=<token>** query parameter.

Connection example (Dart):

```
final hubConnection = HubConnectionBuilder()
  .withUrl(
    'https://your-api.azurewebsites.net/hubs/todo',
    options: HttpConnectionOptions(
      accessTokenFactory: () async => await tokenStorage.getAccessToken(),
    ),
  )
  .build();
```

Server → Client Events

Event	Arguments	Trigger
ReceiveNotification	(String title, String message)	General notification pushed to the user
TaskShared	(String taskId, String taskTitle)	A task was shared with the current user
TaskUpdated	(String taskId)	A shared task was updated or toggled
TransferRequested	(String requestId, String taskTitle)	An ownership transfer request was sent to the current user

Listening example (Dart):

```
hubConnection.on('ReceiveNotification', (args) {
  final title = args?[0] as String;
  final message = args?[1] as String;
  // show in-app notification
});

hubConnection.on('TaskShared', (args) {
  final taskId = args?[0] as String;
  // refresh task list
});
```

Data Schemas

AuthResponse

```
{
  "userId": "string (uuid)",
  "email": "string",
  "token": "string (JWT)",
  "refreshToken": "string",
  "requiresTwoFactor": "boolean",
  "twoFactorToken": "string | null"
}
```

TodoItemResponse

```
{
  "id": "string (uuid)",
  "title": "string",
  "description": "string | null",
  "dueDate": "string (ISO 8601) | null",
  "status": "string",
  "priority": "string",
  "todoListId": "string (uuid) | null",
  "ownerId": "string (uuid)",
  "isOwner": "boolean",
  "completedByUserId": "string (uuid) | null",
  "completedAt": "string (ISO 8601) | null",
  "createdAt": "string (ISO 8601)",
  "updatedAt": "string (ISO 8601) | null",
  "isDeleted": "boolean",
  "deletedAt": "string (ISO 8601) | null",
  "subTasks": "SubTaskResponse[]",
  "tags": "TagResponse[]",
  "sharedWith": "SharedUserResponse[]"
}
```

`status` and `priority` are returned as their **string names** (e.g., `"Open"`, `"High"`), not integers.

SubTaskResponse

```
{
  "id": "string (uuid)",
  "taskId": "string (uuid)",
  "title": "string",
  "status": "string",
  "createdAt": "string (ISO 8601)",
  "updatedAt": "string (ISO 8601) | null"
}
```

TagResponse

```
{
  "id": "string (uuid)",
  "name": "string",
  "createdAt": "string (ISO 8601)"
}
```

TodoListResponse

```
{
  "id": "string (uuid)",
  "name": "string",
  "colorCode": "string | null",
  "ownerId": "string (uuid)",
  "createdAt": "string (ISO 8601)",
  "updatedAt": "string (ISO 8601) | null"
}
```

SharedUserResponse

```
{
  "userId": "string (uuid)",
  "email": "string",
  "sharedAt": "string (ISO 8601)"
}
```

```
}
```

TransferRequestResponse

```
{  
  "id": "string (uuid)",  
  "taskId": "string (uuid)",  
  "taskTitle": "string",  
  "fromUserId": "string (uuid)",  
  "fromUserEmail": "string",  
  "toUserId": "string (uuid)",  
  "toUserEmail": "string",  
  "status": "string",  
  "createdAt": "string (ISO 8601)",  
  "respondedAt": "string (ISO 8601) | null"  
}
```

TodoItemActivityResponse

```
{  
  "id": "string (uuid)",  
  "userId": "string (uuid)",  
  "userEmail": "string",  
  "action": "string",  
  "details": "string",  
  "createdAt": "string (ISO 8601)"  
}
```

TwoFactorEnableResponse

```
{  
  "secret": "string",  
  "qrCodeUri": "string (otpauth URI)"  
}
```